Computational Sociology

Supervised Text Classification

Dr. Thomas Davidson

University of North Carolina at Chapel Hill

October 23, 2026

Plan

1. Course updates
2. Introduction to supervised text classification
3. Supervised text classification in R
4. Data sampling and annotation
5. Biased predictions

Course updates

Project progress report

- 2-3 page double-spaced update on your final project
  - Data collection and analysis
  - Descriptive statistics and preliminary findings
    - At least one table or figure
  - Describe any significant changes
  - Mention any blockers or questions before final version
- Include a short statement describing how you used AI tools
- Submit PDF via email by Friday, November 6

Course updates

Homework

- Homework 2 grades released
- Homework 3 released next week, due Friday, November 13
  - Supervised machine learning
  - Supervised text classification
  - Large language models

Part I: Introduction to supervised text classification

Introduction to supervised text classification

Procedure

- Supervised text classification uses machine learning to automatically label documents
  - Phase 1: Select a corpus of documents
  - Phase 2: Annotate a small sample of documents with "ground truth" labels
  - Phase 3: Train a model to predict the labels of the annotated documents
  - Phase 4: Use the trained model to predict the labels for the entire corpus

Introduction to supervised text classification

Supervised versus unsupervised approaches

- Both supervised text classification and topic modeling can be used as a replacement for conventional content analysis
  - Topic modeling is an *inductive* approach, useful for summarizing an entire corpus and deriving categories
  - Supervised machine learning is a *deductive* approach, designed to classify documents into discrete (or probabilistic) classes based on predefined categories
    - Unlike topic modeling, the categories are determined in advance by the analyst

Introduction to supervised text classification

Approaches to text analysis

- Nelson et al. (2021) compare four approaches to text analysis:
  1. Handcoding
  2. Dictionaries
  3. Unsupervised learning
  4. Supervised learning
- What computational approach can best replicate handcoding?

Introduction to supervised text classification

Approaches to text analysis

- Handcoding is the ideal approach but difficult to scale to large corpora
- Dictionaries
  - Small dictionaries can have high precision but low recall
  - Large dictionaries can have high recall but low precision
- Unsupervised learning
  - Topic modeling and similar approaches are more useful for exploratory analyses, as there is no guarantee that clusters/topics map onto the concept of interest
- Supervised learning
  - Automatically reproduce handcoding at scale without needing a predefined dictionary
  - Important features are learned from the data

Introduction to supervised text classification

Handcoding, dictionaries, and supervised learning

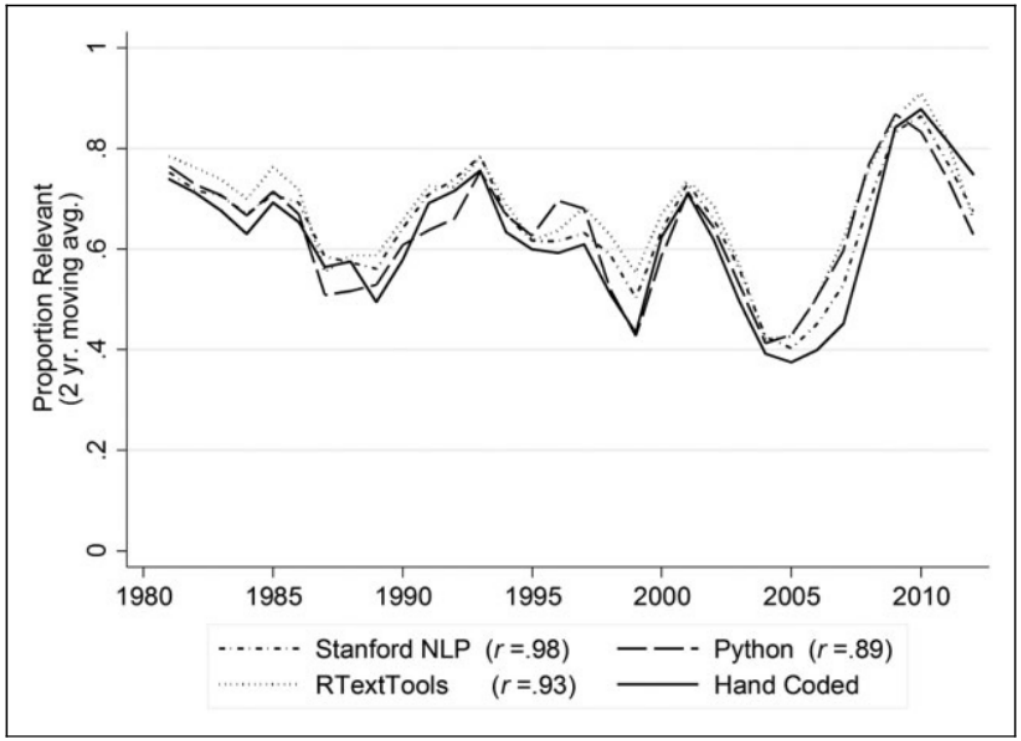
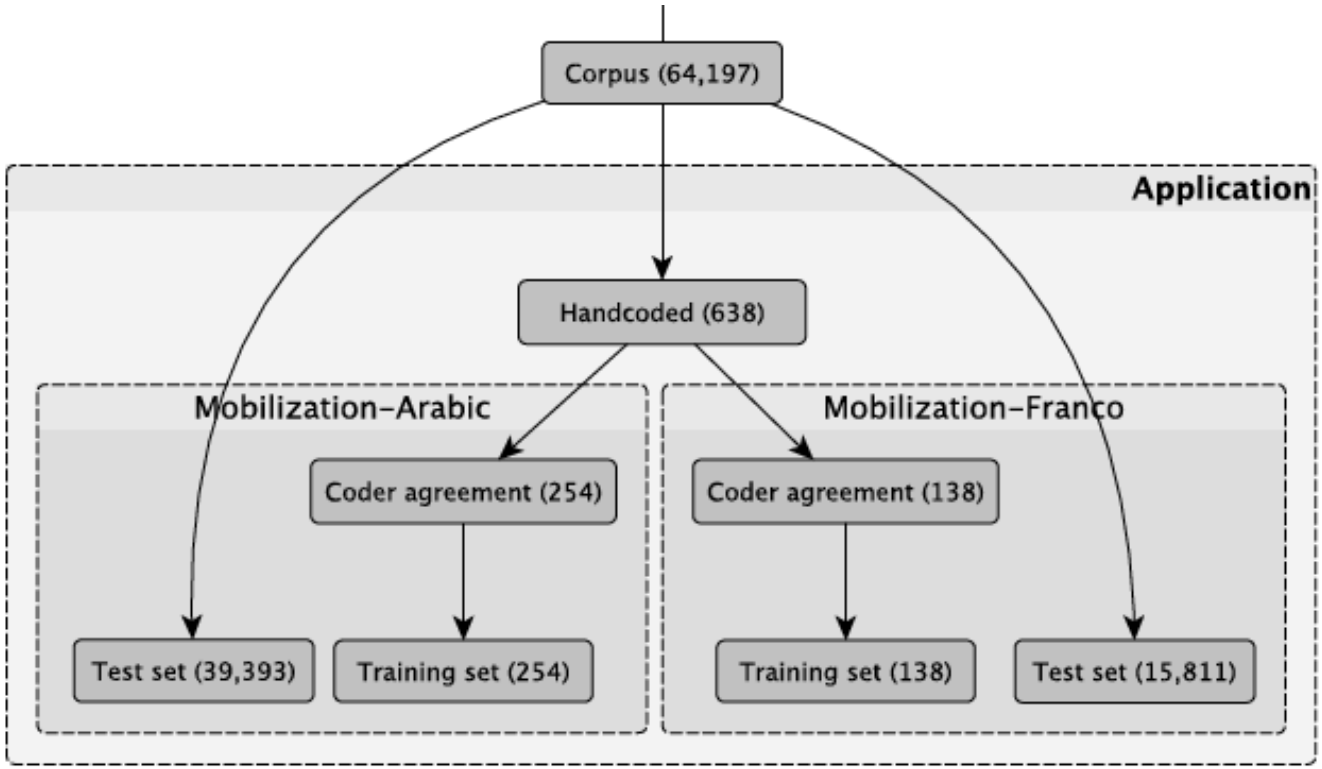


Figure 3. Trends in supervised machine learning analysis of hand-coded articles for relevant versus irrelevant binary scheme (combined relevant substantive categories versus irrelevant category; combined relevant substantive categories shown).

Nelson et al. 2021

Introduction to supervised text classification

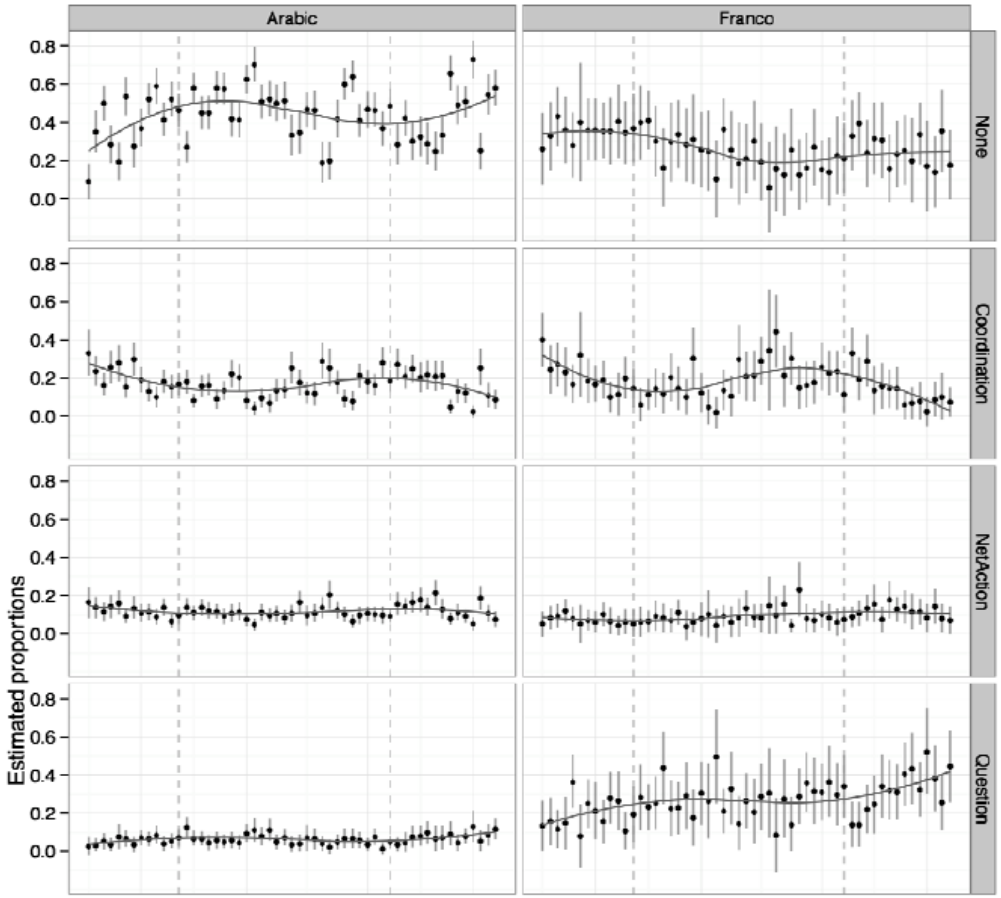
Sociological applications: Social movement framing



Hanna 2013

Introduction to supervised text classification

Sociological applications: Social movement framing



Hanna 2013

Introduction to supervised text classification

Sociological applications: Media frames

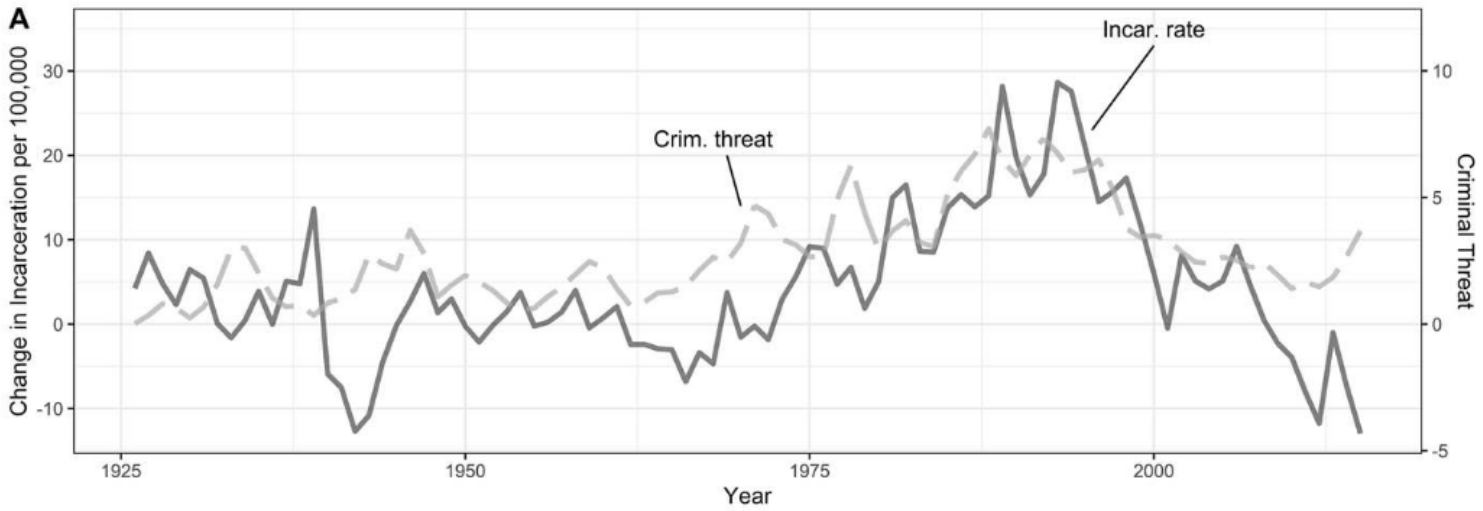
Table 3. Supervised Machine Learning Validation Results

Themes	Accuracy	Precision	Recall	F1 score
Economic threat	0.96	0.79	0.80	0.79
Political threat	0.95	0.88	0.89	0.87
Criminal threat	0.94	0.83	0.86	0.84

Duxbury 2023

Introduction to supervised text classification

Sociological applications: Media frames



Duxbury 2023

Part II: Supervised text classification in R

Supervised text classification in R

Case study

- Data
  - A subsample of the IMDB reviews dataset
    - 5000 IMDB reviews
    - Half positive ($\geq 7/10$), half negative ($\leq 4/10$), neutral excluded
- Classification task
  - Predict which reviews are positive and which are negative (binary)
  - Assumption: The classifier learns the sentiment of the reviews

Maas, Andrew L., Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. 2011. "Learning Word Vectors for Sentiment Analysis." In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, 142–50.

Supervised text classification in R

Loading data

A random sample of the IMDB reviews keeps the process tractable.

```
library(tidyverse)
path <- '../data/imdb_reviews_sample.tsv'
d <- read_tsv(path)

d |> count(sentiment)
```

```
## # A tibble: 2 x 2
##   sentiment     n
##   <chr>     <int>
## 1 neg        2524
## 2 pos        2476
```

Code based on examples from Emil Hvitfeldt and Julia Silge, *Supervised Machine Learning for Text Analysis in R*, Chapter 7

Supervised text classification in R

Train-test split

Divide the data into training and test sets.

```
library(tidymodels)
set.seed(76543)

review_split <- initial_split(d, strata = "sentiment",
                              prop = 0.8)
train <- training(review_split)
test <- testing(review_split)
```

Supervised text classification in R

Creating a `recipe`

A `recipe` specifies the equation and the dataset.

```
review_recipe <- recipe(sentiment ~ text, data = train)
```

Supervised text classification in R

Adding preprocessing steps

The `textrecipes` package adds text preprocessing steps to the recipe. We could also do our preferred preprocessing first using `tidytext` or another package, then pass the resulting data to the recipe.

```
# install.packages("textrecipes")
library(textrecipes)
review_recipe <- review_recipe |>
  step_tokenize(text) |>
  step_tokenfilter(text, max_tokens = 1000) |>
  step_tfidf(text)
```

Supervised text classification in R

Creating a `workflow`

A `workflow` chains together a sequence of modeling operations.

```
review_wf <- workflow() |> add_recipe(review_recipe)
```

Supervised text classification in R

Adding a model

Add a logistic regression with a LASSO penalty to the workflow.

```
# install.packages("glmnet")
lasso <- logistic_reg(penalty = 0.01, mixture = 1) |>
  set_mode("classification") |>
  set_engine("glmnet")

review_wf <- review_wf |> add_model(lasso)
```

Supervised text classification in R

Reviewing the `workflow`

Print the workflow to review the sequence of operations.

```
print(review_wf)
```

```
## == Workflow ============================================================================
## Preprocessor: Recipe
## Model: logistic_reg()
##
## -- Preprocessor ----------------------------------------------------------------------
## 3 Recipe Steps
##
## * step_tokenize()
## * step_tokenfilter()
## * step_tfidf()
##
## -- Model -----------------------------------------------------------------------------
## Logistic Regression Model Specification (classification)
##
## Main Arguments:
##   penalty = 0.01
##   mixture = 1
##
## Computational engine: glmnet
```

Supervised text classification in R

Cross-validation

Last week we compared models on the test set, so the test set was used for model selection. Cross-validation compares models using only the training data. Here we use k-fold (or v-fold) cross-validation with 5 folds.

```
review_folds <- vfold_cv(train, v = 5)
```

Supervised text classification in R

Fitting a cross-validated model

`fit_resamples` from the `tune` package fits the workflow five times, each time holding out one fold for evaluation. `control` specifies information to store for further analysis.

```
review_metrics <- metric_set(precision, recall, f_meas, roc_auc)

fitted <- fit_resamples(
  review_wf,
  review_folds,
  control = control_resamples(save_pred = TRUE),
  metrics = review_metrics
)
```

Supervised text classification in R

Evaluating overall performance

`collect_predictions` from the `tune` package returns the predictions for each held-out fold, and `collect_metrics` summarizes performance across folds.

```
lasso_pred_probs <- collect_predictions(fitted)

collect_metrics(fitted)
```

```
## # A tibble: 4 x 6
##   .metric   .estimator  mean     n std_err .config
##   <chr>     <chr>      <dbl> <int>   <dbl> <chr>
## 1 f_meas    binary     0.829     5 0.00604 pre0_mod0_post0
## 2 precision binary     0.844     5 0.00822 pre0_mod0_post0
## 3 recall    binary     0.815     5 0.00641 pre0_mod0_post0
## 4 roc_auc   binary     0.907     5 0.00375 pre0_mod0_post0
```

`yardstick` treats the first factor level, `neg`, as the outcome (`event`), so precision and recall refer to negative reviews.

Supervised text classification in R

Evaluating performance by fold

Grouping on `id` shows the estimate for each fold.

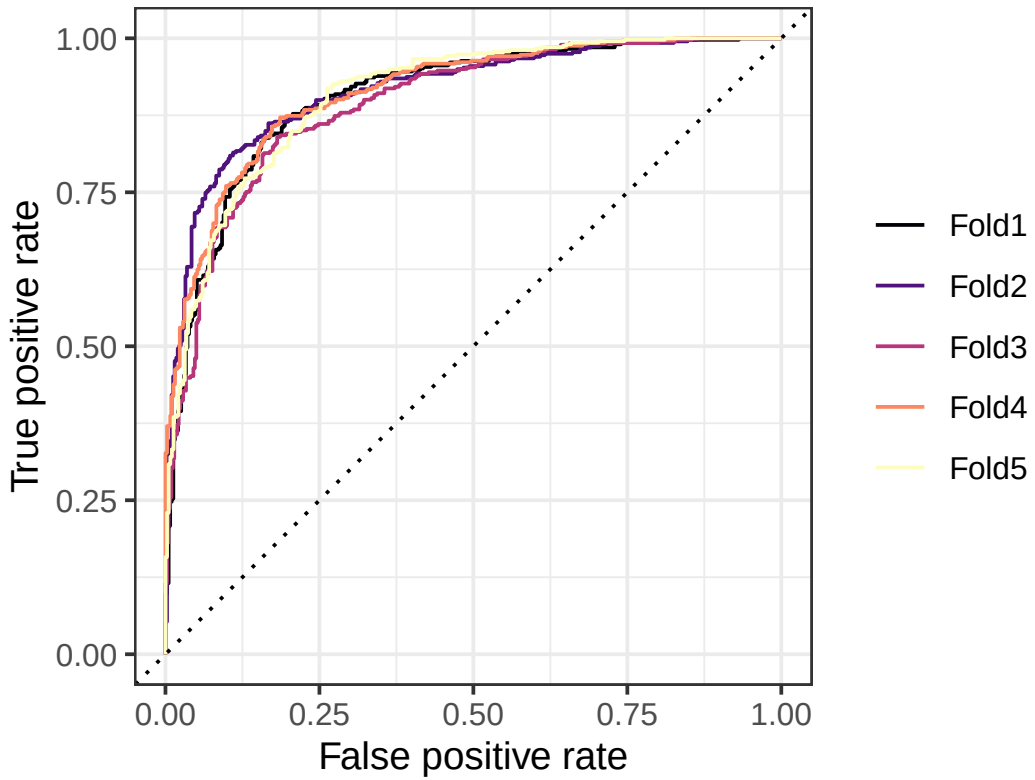
```
lasso_pred_probs |>
  group_by(id) |>
  f_meas(sentiment, .pred_class) |>
  select(id, .estimate)
```

```
## # A tibble: 5 x 2
##   id    .estimate
##   <chr>     <dbl>
## 1 Fold1     0.838
## 2 Fold2     0.844
## 3 Fold3     0.817
## 4 Fold4     0.835
## 5 Fold5     0.813
```

Supervised text classification in R

Computing an ROC curve

The ROC curves show the performance of each classifier. In general they all appear to perform quite well.



Supervised text classification in R

Selecting tuning parameters

Next, find the optimal size of the feature matrix and regularization strength. `tune()` marks these parameters in the recipe and the model.

```
review_recipe_2 <- recipe(sentiment ~ text, data = train) |>
  step_tokenize(text) |>
  step_tokenfilter(text, max_tokens = tune()) |>
  step_tfidf(text)

review_wf <- review_wf |> update_recipe(review_recipe_2)

lasso_2 <- logistic_reg(penalty = tune(), mixture = 1) |>
  set_mode("classification") |>
  set_engine("glmnet")

review_wf <- review_wf |> update_model(lasso_2)
```

Supervised text classification in R

Specifying a parameter grid

`grid_regular` defines the parameter space: a range of values for each parameter and the number of values to test in each range.

```
param_grid <- grid_regular(
  penalty(range = c(-5, 1)),
  max_tokens(range = c(1000, 3000)),
  levels = c(penalty = 5, max_tokens = 3)
)
```

`penalty` ranges are on the $\log_{10}$ scale, so `c(-5, 1)` spans 10^{-5} to 10.

Specifying a parameter grid

```
print(param_grid, n = 15)
```

```
## # A tibble: 15 x 2
##       penalty max_tokens
##         <dbl>      <int>
##  1  0.00001         1000
##  2  0.000316        1000
##  3  0.01            1000
##  4  0.316           1000
##  5 10               1000
##  6  0.00001         2000
##  7  0.000316        2000
##  8  0.01            2000
##  9  0.316           2000
## 10 10               2000
## 11  0.00001         3000
## 12  0.000316        3000
## 13  0.01            3000
## 14  0.316           3000
## 15 10               3000
```

Supervised text classification in R

Fitting the model to the parameter grid

`tune_grid` fits the workflow at each combination of parameters, using the same cross-validation folds as above. There are 15 combinations and 5 folds, so 75 models are evaluated.

```
tune_params <- tune_grid(
  review_wf,
  review_folds,
  grid = param_grid,
  metrics = review_metrics
)
```

Supervised text classification in R

Evaluating performance by parameter

```
f1_scores <- collect_metrics(tune_params) |>
  filter(.metric == "f_meas")

f1_scores |> summarize(mean_f1 = mean(mean), .by = penalty)
```

```
## # A tibble: 5 x 2
##     penalty mean_f1
##       <dbl>   <dbl>
## 1  0.00001    0.792
## 2  0.000316   0.802
## 3  0.01       0.833
## 4  0.316      0.671
## 5 10          0.671
```

Supervised text classification in R

Evaluating performance by parameter

```
f1_scores |> summarize(mean_f1 = mean(mean), .by = max_tokens)
```

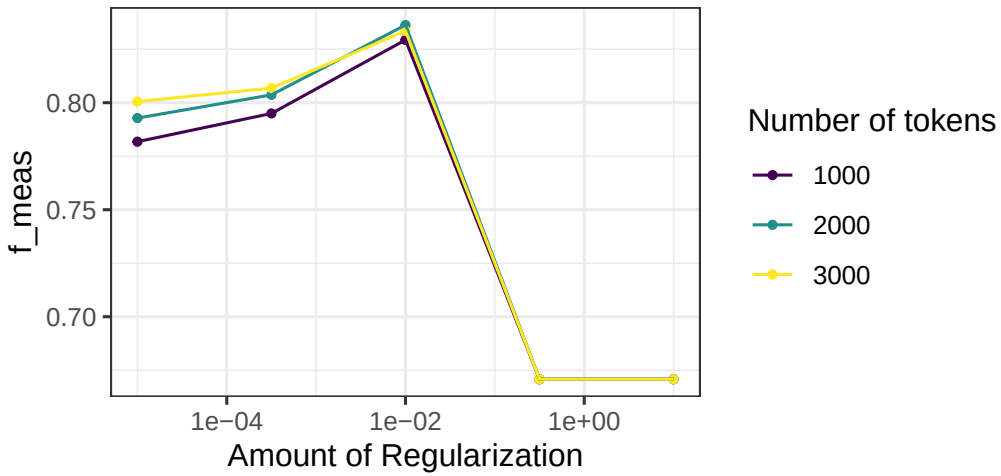
```
## # A tibble: 3 x 2
##   max_tokens mean_f1
##        <int>   <dbl>
## 1       1000   0.750
## 2       2000   0.755
## 3       3000   0.757
```

Supervised text classification in R

Plotting the results

Regularization makes a much bigger difference than the size of the feature matrix. At the two largest penalties every coefficient is zero and the model predicts that every review is negative.

```
autoplot(tune_params, metric = "f_meas") +
  labs(color = "Number of tokens") + scale_color_viridis_d() +
  theme_bw()
```



Supervised text classification in R

Selecting the best model and updating the workflow

Select the parameters with the best F1 score and add them to the workflow.

```
best_f1 <- tune_params |> select_best(metric = "f_meas")
print(best_f1)
```

```
## # A tibble: 1 x 3
##   penalty max_tokens .config
##     <dbl>      <int> <chr>
## 1    0.01       2000 pre2_mod3_post0
```

```
final_wf <- finalize_workflow(review_wf, best_f1)
```

Supervised text classification in R

Fitting the model to the training data

`last_fit` fits the final model to the entire training set *and* evaluates it on the test set, which has not been used so far. There is no cross-validation at this stage: the best model is retrained using all of the training data.

```
final_fitted <- last_fit(final_wf, review_split,
                         metrics = review_metrics)
```

Supervised text classification in R

Evaluating out-of-sample performance

```
collect_metrics(final_fitted)
```

```
## # A tibble: 4 x 4
##   .metric   .estimator .estimate .config
##   <chr>     <chr>          <dbl> <chr>
## 1 precision binary         0.868 pre0_mod0_post0
## 2 recall    binary         0.824 pre0_mod0_post0
## 3 f_meas    binary         0.846 pre0_mod0_post0
## 4 roc_auc   binary         0.919 pre0_mod0_post0
```

Supervised text classification in R

Evaluating out-of-sample performance

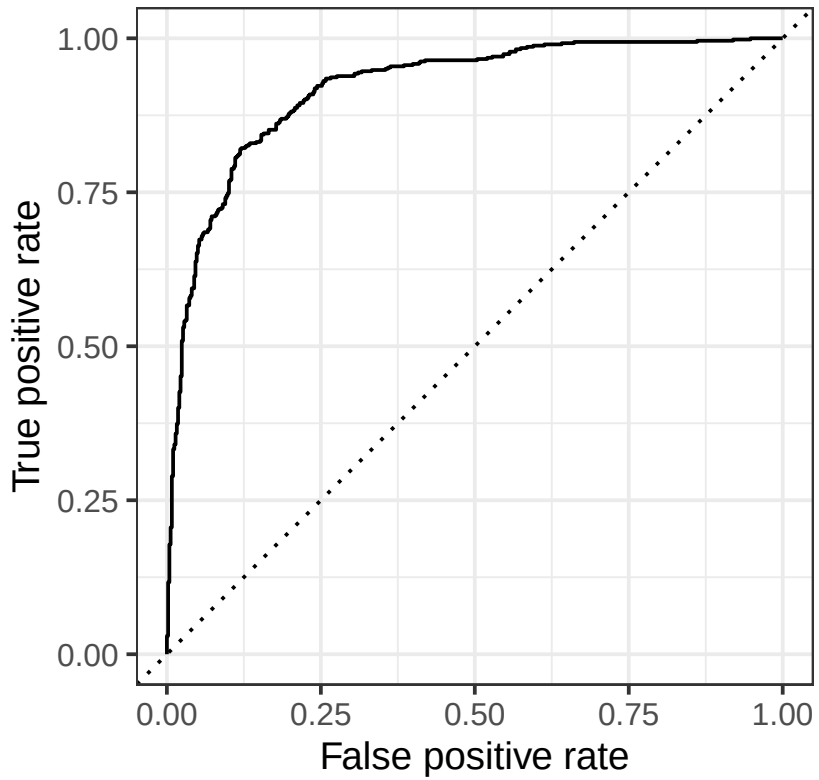
```
collect_predictions(final_fitted) |>
  conf_mat(truth = sentiment, estimate = .pred_class)
```

```
##           Truth
## Prediction neg pos
##        neg 416  63
##        pos  89 433
```

Supervised text classification in R

Evaluating out-of-sample performance

The ROC curve for the held-out data shows that the model performs well out of sample.



Supervised text classification in R

Error analysis

The final step is to analyze the predictions. It's often most insightful to look at the errors. `augment` returns the test data with the predictions.

```
reviews_pred <- augment(final_fitted)
```

Supervised text classification in R

Error analysis: Positive reviews predicted negative

```
reviews_pred |>
  filter(sentiment == "pos", .pred_pos < 0.3) |>
  select(text) |>
  slice_sample(n = 5)
```

```
## # A tibble: 5 x 1
##   text
##   <chr>
## 1 The young lady's name is Bonnie (Polay). She's attractive, is~
## 2 This is about a mad scientist who creates a half shark - half~
## 3 I think this movie got a low rating because it got judged by ~
## 4 I didn't even want to watch this movie after reading Maltin's~
## 5 I just don't understand why this movie is getting beat-up in ~
```

Supervised text classification in R

Error analysis: Negative reviews predicted positive

```
reviews_pred |>
  filter(sentiment == "neg", .pred_neg < 0.3) |>
  select(text) |>
  slice_sample(n = 5)
```

```
## # A tibble: 5 x 1
##   text
##   <chr>
## 1 "This woman never stops talking throughout the movie. She mem~
## 2 "I can only believe that Garson Kanin must have been two peop~
## 3 "If I had realized John Wayne was in this movie, I would not ~
## 4 "I like Brad Pitt enormously. He is an actor with brains and ~
## 5 "Devil Dog sets your heart racing. It's brilliantly paced, th~
```

Supervised text classification in R

Inspecting the coefficients

The LASSO coefficients show which words are most strongly associated with each class. Positive coefficients increase the predicted probability of pos.

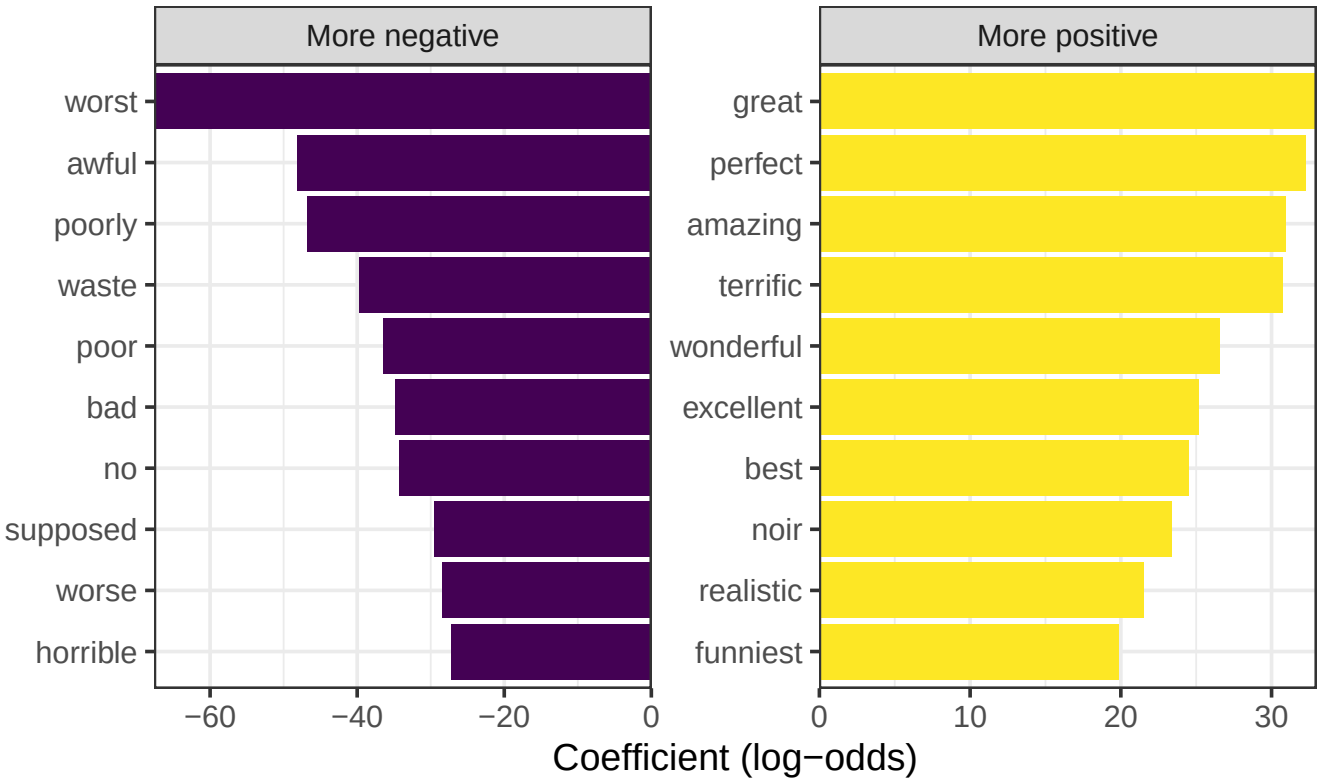
```
coefs <- extract_fit_parsnip(final_fitted) |>
  tidy() |>
  filter(term != "(Intercept)") |>
  mutate(
    term = str_remove(term, "tfidf_text_"),
    sign = if_else(estimate > 0, "More positive", "More negative"))

coefs |> count(nonzero = estimate != 0)
```

```
## # A tibble: 2 x 2
##   nonzero     n
##   <lgl>   <int>
## 1 FALSE    1600
## 2 TRUE      400
```

Supervised text classification in R

Visualizing the coefficients



Coefficients are changes in the log-odds of positive class per one-unit increase in TF-IDF.

Supervised text classification in R

Further improvements

- Test different algorithms and parameters
  - SVM and neural networks both work well for many text classification problems
- Alternative feature representations
  - N-grams
  - Embeddings (LSA, word2vec, GloVe, BERT)
  - Metadata
- Evaluate the impact of different preprocessing techniques (Denny and Spirling 2018)

Part III: Data sampling and annotation

Data sampling and annotation

Overview

- We used a dataset with "ground truth" or "gold standard" labels derived from the numeric scores given to movies by IMDB users

Data sampling and annotation

Overview

- In practice, we often have to create an annotated dataset before we can train a model
- There are many decisions we have to make, including:
  - What data do we want to classify?
  - What are the classes we will code for and what is the coding scheme?
  - Which texts should we annotate?
  - How many annotated examples do we need?
  - Who performs the annotation?

Data sampling and annotation

Data selection and coding scheme

- The choice of data and coding scheme is usually driven by a combination of substantive and theoretical concerns
- Consider the following example from my research on hate speech:
  - Sampling frame: 25k tweets containing keywords from a crowdsourced hate speech database
  - Classes:
    - Hate speech
    - Offensive language
    - Neither

Data sampling and annotation

Data selection and coding scheme

- Codebook development to identify a meaningful set of categories by reading documents
- Clear definitions and examples
- Intercoder reliability checks

Data sampling and annotation

Data selection and coding scheme

- Generalizability:
  - Context (e.g., publication, platform)
  - Period
  - Language / dialect
  - Medium (e.g., text, image, video)
- Heuristic: Classifier trained on one domain likely less accurate (or entirely wrong) on other domains or tasks
  - Example: Sentiment $\neq$ Stance

Bestvater, Samuel E., and Burt L. Monroe. 2023. "Sentiment Is Not Stance: Target-Aware Opinion Classification for Political Text Analysis." *Political Analysis* 31(2): 235–56.

Data sampling and annotation

The unit of analysis

- Select an appropriate unit of analysis
  - A book, a chapter, a paragraph, a sentence?
- This is usually guided by theory
  - e.g., if we want to classify the political leaning of a comment, then the unit is the comment, but if we want to classify users, then we might consider aggregating their comments
- Empirical evaluations
  - Limited benefits to classifying sentences rather than chunks of newspaper articles, but this may not generalize to other cases (Barberá et al. 2021)

Data sampling and annotation

Keywords and classes

- Keywords often used to sample documents for analysis
  - Social media posts
  - Newspaper articles
  - Academic articles
  - Archival documents

Data sampling and annotation

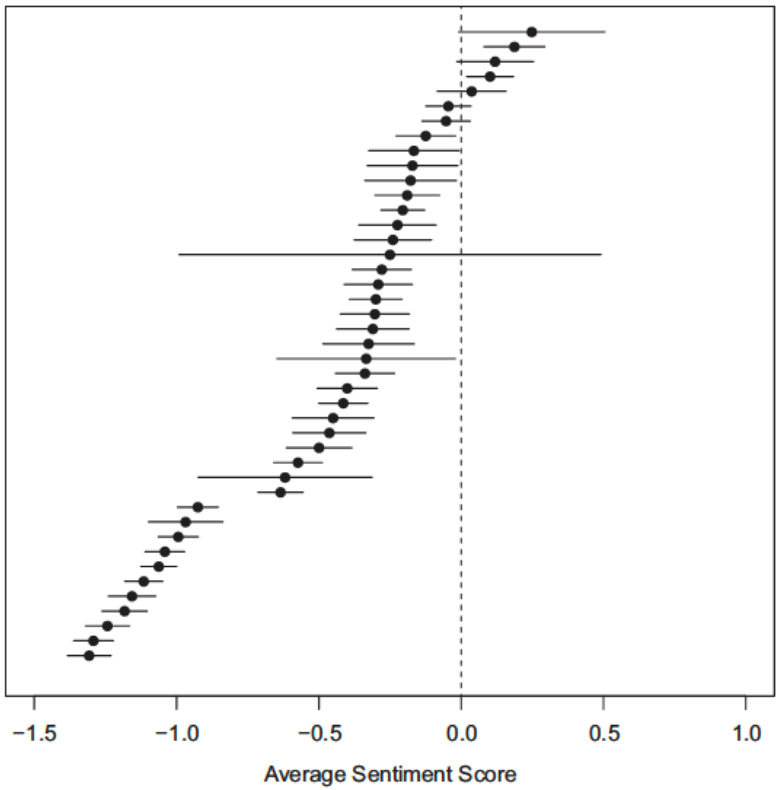
Keywords and classes



King, Lam, and Roberts 2017

Data sampling and annotation

Keywords and classes



Note: Each document set was selected by a different keyword list, with point estimates (as dots) and 95% confidence intervals (horizontal lines) shown.

King, Lam, and Roberts 2017

Data sampling and annotation

Keywords and classes

- Recommendations
  - Start small, ~1-5 high-precision keywords
  - Use an automated discovery approach* and/or domain expertise to expand the keyword set
  - Increase the size of the keyword set as long as returned documents increase but relevant proportion does not decline (Barberá et al. 2021)

* Python code to implement the KLR approach.

Data sampling and annotation

Sampling for annotation

- The goal is to annotate a sample of documents that can train a classifier that generalizes well beyond the training data
- Typically entails using random sampling or stratified random sampling

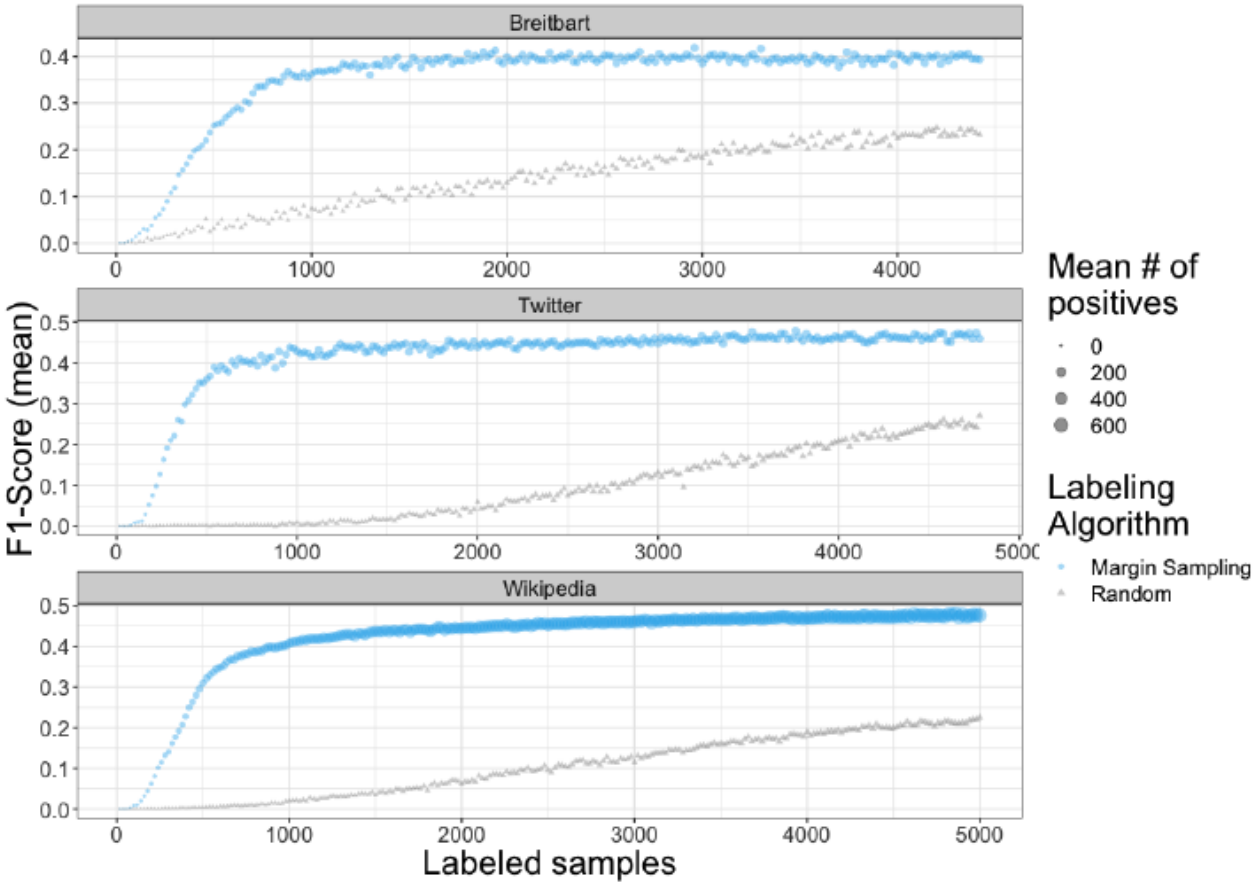
Data sampling and annotation

Sampling for annotation

- Random sampling can be inefficient
  - Duplicate or highly similar documents are redundant
  - Many documents do not provide any new information
  - Insufficient coverage of rare classes
- We can do better by using an approach called *active learning*

Data sampling and annotation

Active learning in practice



Miller, Linder, and Mebane 2020

Data sampling and annotation

Number of annotated documents

- Context specific
  - How balanced are the class distributions?
    - e.g., evenly split like IMDB, or are some classes rare?
  - How complex is the classification task?
    - Number of categories
    - Task difficulty, e.g., requires expertise; subjective
- Heuristic
  - *Required sample size increases with imbalance, complexity, and task difficulty*

Data sampling and annotation

Number of annotated documents

- Empirical results
  - Diminishing returns in a newspaper content classification task after ~1000 examples (Barberá et al. 2021)
  - Active learning can substantially reduce the necessary sample size compared to random sampling (Miller, Linder, and Mebane 2020)
  - Large language models can drastically reduce the number of annotated documents needed (more next week)

Data sampling and annotation

Annotation process

- Who should do the coding?
  - Expert coders vs. research assistants vs. crowdworkers
- How many coders per data point?
  - Conventional to have multiple coders to ensure reliability
  - Better to have more examples coded by fewer people than fewer examples coded by more people, but some overlap needed for intercoder reliability (Barberá et al. 2021)

Data sampling and annotation

Sampling and annotation bias

- Sampling biases
  - Over/undersampling
  - Selection bias
- Annotation biases
  - Stereotypes and beliefs
  - Domain knowledge

Davidson 2025

Part IV: Biased predictions

Biased predictions

Digital audit studies of text classifiers

How it Works

Using Machine Learning to Reduce Toxicity Online

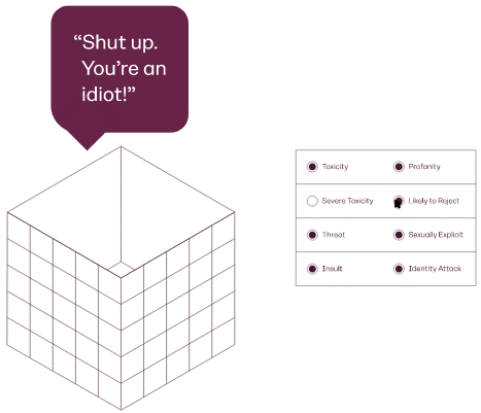
Perspective uses machine learning models to identify abusive comments. The models score a phrase based on the perceived impact the text may have in a conversation. Developers and publishers can use this score to give feedback to commenters, help moderators more easily review comments, or help readers filter out "toxic" language.

Perspective models provide scores for several different attributes. In addition to the flagship Toxicity attribute, here are some of the other attributes Perspective can provide scores for:

- Severe Toxicity
- Identity attack
- Insult
- Threat
- Profanity
- Sexually explicit

To learn more about our ongoing research and experimental models, visit our Developers site.

LEARN MORE



https://perspectiveapi.com/how-it-works/. Perspective API is being retired, and the service ends after 2026.

Biased predictions

Digital audit studies of text classifiers

Term	Toxic	Overall
atheist	0.09%	0.10%
queer	0.30%	0.06%
gay	3%	0.50%
transgender	0.04%	0.02%
lesbian	0.10%	0.04%
homosexual	0.80%	0.20%
feminist	0.05%	0.05%
black	0.70%	0.60%
white	0.90%	0.70%
heterosexual	0.02%	0.03%
islam	0.10%	0.08%
muslim	0.20%	0.10%
bisexual	0.01%	0.03%

Table 1: Frequency of identity terms in toxic comments and overall.

Dixon, Lucas, John Li, Jeffrey Sorensen, Nithum Thain, and Lucy Vasserman. 2018. "Measuring and Mitigating Unintended Bias in Text Classification." In *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society*, 67–73.

Biased predictions

Digital audit studies of text classifiers

Template Examples	Label
I am <IDENTITY>	Non-Toxic
I am a <IDENTITY> person, ask me anything	Non-Toxic
<IDENTITY> people are just like everyone else	Non-Toxic
I hate all <IDENTITY>	Toxic
I am a <IDENTITY> person and I hate your guts and think you suck	Toxic
<IDENTITY> people are gross and universally terrible	Toxic

Table 2: Phrase template examples.

Dixon et al. 2018

Biased predictions

Digital audit studies of text classifiers

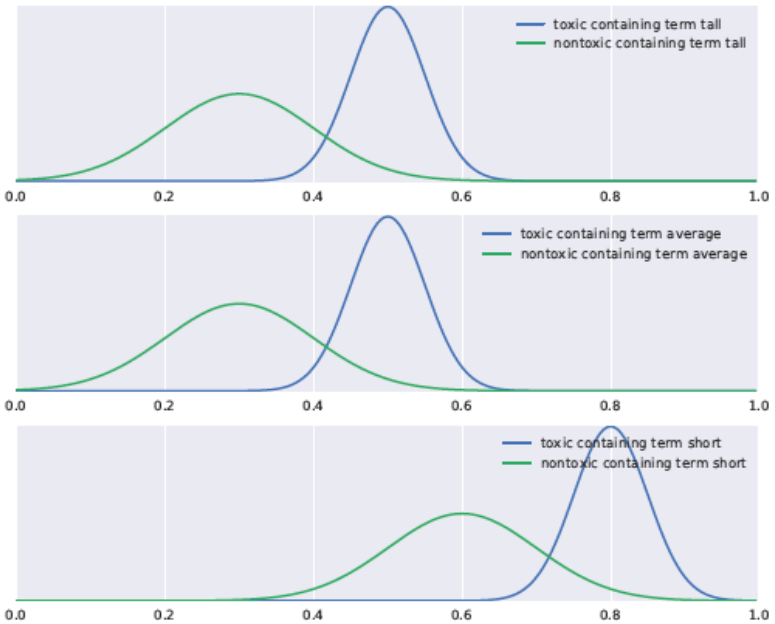
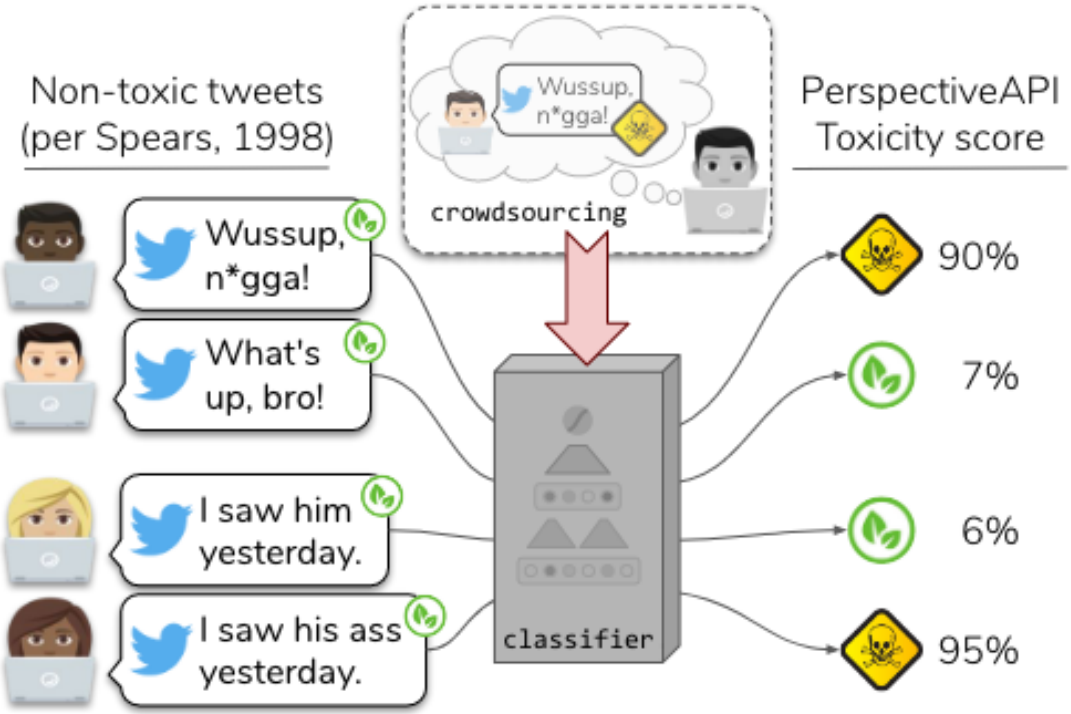


Figure 2: Distributions of toxicity scores for three groups of data, each containing comments with different identity terms, "tall", "average", or "short".

Dixon et al. 2018

Biased predictions

Racial bias in hate speech detection classifiers



Sap, Maarten, Dallas Card, Saadia Gabriel, Yejin Choi, and Noah A. Smith. 2019. "The Risk of Racial Bias in Hate Speech Detection." In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 1668–78.

Biased predictions

Racial bias in hate speech detection classifiers

Dataset	Class	$\widehat{p_{i_{black}}}$	$\widehat{p_{i_{white}}}$	t	p	$\frac{\widehat{p_{i_{black}}}}{\widehat{p_{i_{white}}}}$
Waseem and Hovy	Racism	0.001	0.003	-20.818	***	0.505
	Sexism	0.083	0.048	101.636	***	1.724
Waseem	Racism	0.001	0.001	0.035		1.001
	Sexism	0.023	0.012	64.418	***	1.993
	Racism and sexism	0.002	0.001	4.047	***	1.120
Davidson et al.	Hate	0.049	0.019	120.986	***	2.573
	Offensive	0.173	0.065	243.285	***	2.653
Golbeck et al.	Harassment	0.032	0.023	39.483	***	1.396
Founta et al.	Hate	0.111	0.061	122.707	***	1.812
	Abusive	0.178	0.080	211.319	***	2.239
	Spam	0.028	0.015	63.131	***	1.854

Davidson, Thomas, Debasmita Bhattacharya, and Ingmar Weber. 2019. "Racial Bias in Hate Speech and Abusive Language Detection Datasets." In *Proceedings of the Third Workshop on Abusive Language Online*, 25–35.

Biased predictions

Racial bias in hate speech detection classifiers

Dataset	Class	$\widehat{p_{i_{black}}}$	$\widehat{p_{i_{white}}}$	t	p	$\frac{\widehat{p_{i_{black}}}}{\widehat{p_{i_{white}}}}$
Waseem and Hovy	Racism	0.010	0.010	-0.632		0.978
	Sexism	0.963	0.944	20.064	***	1.020
Waseem	Racism	0.011	0.011	-1.254		0.955
	Sexism	0.349	0.290	28.803	***	1.203
	Racism and sexism	0.012	0.012	-0.162		0.995
Davidson et al.	Hate	0.017	0.015	4.698	***	1.152
	Offensive	0.988	0.991	-6.289	***	0.997
Golbeck et al.	Harassment	0.099	0.091	6.273	***	1.091
Founta et al.	Hate	0.074	0.027	46.054	***	2.728
	Abusive	0.925	0.968	-41.396	***	0.956
	Spam	0.010	0.010	0.000		1.000

This table shows results for tweets containing the word "b***h". The term was used in ~1.7% of AAE and 0.5% of SAE tweets.

Biased predictions

Racial bias in hate speech detection classifiers

Threatening language	**F5**: Direct threat	"I am going to shoot the next [IDENTITY] I see."	hateful
	F6: Threat as normative statement	"We should get rid of [IDENTITY] once and for all."	hateful
Slur usage	**F7**: Hate expressed using slur	"You are just a [SLUR] to me."	hateful
	F8: Non-hateful homonyms of slurs	"I had this queer feeling we were being watched."	non-hate
	F9: Reclaimed slurs	"I'm the freshest n*gga in the room."	non-hate
Profanity usage	**F10**: Hate expressed using profanity	"These [IDENTITY] should shut the f*ck up once and for all."	hateful
	F11: Non-hateful use of profanity	"I wish someone could get me the f*ck out of here."	non-hate

Röttger, Paul, Bertie Vidgen, Dong Nguyen, Zeerak Waseem, Helen Margetts, and Janet Pierrehumbert. 2021. "HateCheck: Functional Tests for Hate Speech Detection Models." In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, 41–58.

Concluding remarks

- Data sampling and annotation is an overlooked area of research but has a huge impact on downstream applications
  - *Garbage in, garbage out*
- There are many different decisions involved that require careful consideration
  - Unlike the machine learning phase, we typically don't have the budget to do a grid search over the parameter space, so it is important to get it right the first time

Summary

- Supervised text classification combines NLP and ML to classify documents into classes
- Training data must be carefully sampled and annotated
- Model features and parameters are selected to maximize predictive performance
- Error analysis and feature importance provide insight into performance
- Models should be evaluated for biases
- Once a model performs well and has been validated out of sample, we use it to predict the remainder of the corpus

Next class

- Introduction to large language models